

Environment and DevOps

Project Name

CarbonWise

Version: 1.0

Status: Approved

Deployment Strategy: Containerized

CI/CD Platform: GitHub Actions

1. Purpose

This document defines the deployment, infrastructure, CI/CD, monitoring, security, and operational strategy for CarbonWise.

The architecture is designed to maximize:

- Code Quality (High Impact)
- Security (Medium Impact)
- Efficiency (Medium Impact)
- Testing Reliability
- Maintainability

while keeping the repository:

Single Branch

Public Repository

Under 10 MB

as required by the challenge.

2. DevOps Objectives

The system must:

- Deploy consistently
- Support automated testing
- Enable rapid iteration
- Maintain security standards
- Provide operational visibility

3. Environment Strategy

Three environments will be maintained.

Development

Purpose:

```text id="n1gmx8"  
Local Development

Used For:

- Feature Development
- Unit Testing
- Debugging

---

## Staging

Purpose:

```text id="evq3o0" Pre-Production Validation

Used For:

- Integration Testing
- Playwright Testing
- Security Verification

Production

Purpose:

```
```text id="7o3t2z"
Live User Environment
```

Used For:

- Real User Traffic
- Monitoring
- Analytics

---

## 4. Technology Stack

### Frontend

```
```text id="61l5fk" Next.js 15
```

TypeScript

Tailwind CSS

Shadcn/UI

```
---

## Backend

```text id="c1r5cz"
NestJS

TypeScript
```

---

### Database

```
```text id="8vqekv" PostgreSQL
```

```
---

## ORM
```

```
```text id="sp7z3e"
Prisma
```

---

## Cache

```
```text id="gjm44y" Redis
```

```
---

# 5. Infrastructure Overview

```text id="h84z0i"
User

↓

Next.js Frontend

↓

NestJS API

↓

Redis Cache

↓

PostgreSQL
```

---

## 6. Local Development Environment

Developers run the system using:

```
```text id="v1q8qj" Docker Compose
```

```
---

## Local Services
```

```
```text id="vlnuq0"
Frontend

Backend

PostgreSQL

Redis
```

## Docker Compose Structure

```yaml id="m14sza" services: frontend: backend: postgres: redis:

```
---

# Benefits

- Consistent Setup
- Easy Onboarding
- Reduced Environment Issues

---

# 7. Containerization Strategy

Each major service runs inside Docker.

---

## Frontend Container

Responsibilities:

- UI Rendering
- Static Assets

---

## Backend Container

Responsibilities:

- APIs
- Authentication
```

```
- Business Logic

---

## Database Container

Responsibilities:

- Persistent Storage

---

## Redis Container

Responsibilities:

- Caching
- Session Data

---

# Example Backend Dockerfile

```dockerfile id="1b4xzw"
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

CMD ["node","dist/main.js"]
```

---

## 8. Git Strategy

Challenge requirement:

```text id="5dyho" Single Branch Only

```
---
```

```
## Branch
```

```
```text id="mp90wy"  
main
```

---

## Workflow

```text id="z5m0qf" Code

↓

Commit

↓

Push

↓

GitHub Actions

↓

Deploy

```
---
```

```
# 9. CI/CD Pipeline
```

```
Platform:
```

```
```text id="uowu5l"  
GitHub Actions
```

---

## Pipeline Overview

```text id="jsbgr8" Push

↓

ESLint

↓

Type Check

↓

Jest

↓

Supertest

↓

Build

↓

Playwright

↓

Deploy

10. Code Quality Automation

Supports:

```text id="rqxfql"

Code Quality (High Impact)

---

## ESLint

Purpose:

- Enforce Consistency
- Prevent Common Mistakes

---

## Prettier

Purpose:

- Consistent Formatting
- 

## TypeScript

Purpose:

- Static Type Safety
- 

## Pipeline Rules

Build fails if:

Lint Errors Exist

Type Errors Exist

Tests Fail

```

11. Testing Automation

Supports:

Testing
```

---

## Unit Tests

Jest

```

Integration Tests
```

```
```text id="4eqn0j"
Supertest
```

E2E Tests

```
```text id="3cqlg4" Playwright
```

```

Trigger

Every Pull Request:

```text id="4pt2e0"
Run Full Test Suite
```

12. Security Pipeline

Supports:

```
```text id="f8ylz9" Security (Medium Impact)
```

```

Dependency Scanning

Tool:

```text id="4crzj9"
npm audit
```

Secret Scanning

Prevent:

```
```text id="pwk20o" API Keys
```

Passwords

Tokens

```
from being committed.

Security Headers

Implemented Using:

```text id="0jlj6w"  
Helmet
```

Helmet Configuration

Provides:

```text id="7u53ow" Content Security Policy

X-Frame-Options

X-Content-Type-Options

```

Rate Limiting

Protect APIs:

```text id="8h7o5w"  
100 Requests / Minute
```

Password Security

Technology:

```text id="fdt4oq" bcrypt

```

JWT Security

Includes:

```text id="lp3x7v"  
Access Token  
  
Refresh Token
```

Environment Variables

Stored In:

```
```text id="1x4b8f" .env
```

```

Never Commit:

```text id="ehht40"  
Secrets  
  
Passwords  
  
API Keys
```

13. Database Operations

Migration Strategy

Tool:

```
```text id="g2r91q" Prisma Migrate
```

```

Workflow
```

```
```text id="pk4th7"
Schema Change
```

↓

Migration

↓

Review

↓

Deploy

Backup Strategy

Daily

Incremental Backup

Weekly

Full Backup

Monthly

Archive Backup

14. Caching Strategy

Supports:

```
```text id="2t7c6l" Efficiency (Medium Impact)
```

```

```

```
Technology
```

```
```text id="wazkg6"
```

```
Redis
```

Cached Resources

Dashboard

Recommendations

Analytics

Carbon Twin Results

Benefits

```
```text id="1g7xiw" Reduced Database Load
```

Faster Responses

Lower Latency

```

```

```
15. Monitoring Strategy
```

```
Purpose:
```

```
Maintain system health.
```

```

```

```
Metrics Tracked
```

```
API Response Time
```

Target:

```
```text id="1v64yu"
<500 ms
```

Dashboard Load Time

Target:

```
```text id="qp7r89" <2 seconds
```

```

```

```
Error Rate
```

Target:

```
```text id="qzd21s"
<1%
```

Uptime

Target:

```
```text id="ujk2yc" 99.9%
```

```

```

```
16. Logging Strategy
```

```
All major events are logged.
```

```

```

```
Authentication Events
```

Examples:

```
```text id="7y3cqj"
```

```
Login
```

```
Logout
```

Failed Login

Business Events

Examples:

```text id="f5lkg9" Goal Creation

Recommendation Acceptance

Carbon Calculation

---

# Error Events

Examples:

```text id="eprsv6"  
Database Errors

Validation Failures

Unhandled Exceptions

17. Performance Optimization

Supports:

```text id="71nq34" Efficiency (Medium Impact)

---

# Database Indexes

Indexes:

```text id="ut5xjt"  
email

```
user_id
created_at
```

Pagination

Applied To:

```
```text id="5n1yxm" Activities
```

Recommendations

Notifications

```

Lazy Loading

Applied To:

```text id="a0v4pa"
Charts

Analytics

Historical Data
```

Query Optimization

Avoid:

```
```sql id="p6z3ci" SELECT *
```

```

Prefer:
```

```
```sql id="s7jdh5"
SELECT id, title
```

18. Accessibility Validation

Supports:

```
```text id="8c7m34" Accessibility (Low Impact)
```

```

Automated Checks

Tools:

```text id="mz2x0f"
Playwright

axe-core
```

Manual Validation

Verify:

- Keyboard Navigation
- Screen Readers
- Focus Indicators

19. Deployment Strategy

Development

Automatic Deployment

Staging

Approval Required

Production

Manual Approval Required

Release Workflow

```text id="6h2r9w" Build

↓

Tests

↓

Security Checks

↓

Deployment

↓

Monitoring

---

# 20. Disaster Recovery

Recovery Goals:

---

## RTO

```text id="eqk8e4"

1 Hour

RPO

text id="wtz7h6"

15 Minutes

21. Operational Checklist

Before Every Release:

- ✓ Lint Passes
 - ✓ Type Check Passes
 - ✓ Jest Passes
 - ✓ Supertest Passes
 - ✓ Playwright Passes
 - ✓ Security Scan Passes
 - ✓ Database Migration Reviewed
 - ✓ No Critical Bugs Open
-

22. Future Infrastructure Expansion

Supports:

- AI Carbon Coach
- Community Challenges
- Smart Device Integration
- Enterprise Sustainability Programs

without architectural redesign.

23. Success Criteria

The DevOps strategy is successful when:

- ✓ Deployments are repeatable
- ✓ Tests run automatically
- ✓ Security controls are enforced
- ✓ Performance targets are met
- ✓ Accessibility checks are included
- ✓ Production incidents are minimized
- ✓ Development remains efficient
- ✓ The platform remains maintainable as it grows